Socket

December 1, 2025

Contents

1 Introduction: Understanding Network Communication 3

2 The Fundamentals of Sockets 3
2.1 The Role of Sockets in the Network Stack . . . 3
2.2 Socket Addresses: IP and Port . . . 3
2.3 Core System Calls . . . 3
2.4 The Client-Server Architecture . . . 4

3 A Comparison of Socket Types 4

4 The C/C++ Socket Programming Toolkit 4
4.1 Core Functions . . . 5
4.1.1 socket() function . . . 5
4.1.2 bind() function . . . 5
4.1.3 close() and shutdown() functions . . . 5
4.2 Address and Byte Order Manipulation . . . 6

5 Implementing UDP Sockets 6
5.1 Key UDP Communication Functions . . . 6
5.2 UDP Server Implementation . . . 6
5.2.1 Server Setup . . . 7
5.2.2 Data Exchange Loop . . . 7
5.3 UDP Client Implementation . . . 7
5.3.1 Client Setup . . . 7
5.3.2 Data Exchange Loop . . . 8

6 Implementing TCP Sockets 8
6.1 Key TCP Communication Functions . . . 8
6.1.1 Server-Side Sequence . . . 8
6.1.2 Client-Side Function . . . 8
6.1.3 Data Transfer Functions . . . 8
6.2 TCP Server Implementation . . . 9
6.2.1 Setup Listening Socket . . . 9
6.2.2 Listen and Accept Connection . . . 9
6.2.3 Data Exchange and Closure . . . 9
6.3 TCP Client Implementation . . . 10
6.3.1 Client Setup . . . 10
6.3.2 Connect and Exchange Data . . . 10

7 Advanced Topics and Best Practices 10
7.1 Buffer Management . . . 11
7.2 Configuring Socket Options . . . 11
7.3 Handling the SIGPIPE Signal . . . 11
7.3.1 The Problem . . . 11
7.3.2 Solution 1: Ignoring the Signal . . . 11
7.3.3 Solution 2: Using a Custom Handler . . . 12

8 Practical Networking Utilities 12
8.1 `ping` . . . 12
8.2 `netcat` (`nc`) . . . 12

1 Introduction: Understanding Network Communication

Socket programming is the fundamental mechanism for enabling communication between programs over a network. **Sockets** provide a standardized application programming interface (API) that allows processes, whether on the same machine or across the world, to exchange data. This guide will serve as a detailed set of university-level notes, breaking down the core concepts of sockets, the primary communication protocols they facilitate (UDP and TCP), and the practical C++ implementation of client-server applications.

This document is structured to build a strong foundational understanding. We will begin with the theoretical underpinnings of what a socket is and its role in the network stack. From there, we will explore the different types of sockets and their specific use cases. We will then detail the essential C++ functions and libraries required for network programming, provide complete code examples for both UDP and TCP applications, and conclude with advanced topics such as error handling, socket configuration, and best practices for building robust network code.

—

2 The Fundamentals of Sockets

Before writing any network code, it is essential to have a solid understanding of what a socket is, its role within the layered network model, and the client-server architecture it enables. This section establishes the conceptual framework necessary for practical application.

A socket is best understood through the Unix philosophy: "everything is a file." In this context, a socket is a special type of **file descriptor**—an integer that represents an open file—used for inter-process communication. Just as you would read from or write to a file on disk, your program can use a socket file descriptor to send and receive data over a network connection.

2.1 The Role of Sockets in the Network Stack

Sockets operate at the **transport layer** of the network stack, facilitating communication between specific processes. This is a critical distinction from the network layer's role.

- **Network Layer**: Responsible for *host-to-host* communication. It ensures that a message is delivered from one machine to the correct destination machine on the network.
- **Transport Layer**: Responsible for *process-to-process* communication. Once a message arrives at the correct host, the transport layer delivers it to the specific application or service (i.e., the process) that is intended to receive it. **Sockets are the mechanism that makes this possible.**

2.2 Socket Addresses: IP and Port

Every socket is defined by a unique combination of an **IP address** and a **port number**.

- `ip_address`: The unique address of the host machine on the network.
- `port`: An integer that identifies the specific process or service on that host.

It is a standard convention to use port numbers greater than 1024 for custom applications. Lower-numbered ports are reserved for well-known services, such as FTP (port 21), SSH (port 22), and HTTP (port 80), to avoid conflicts.

2.3 Core System Calls

Communication through sockets is managed by a set of C-language system routines. The most basic of these are:

- `socket()`: Creates a new socket file descriptor.
- `send()`: Transmits data through the socket.

- `recv()`: Receives data from the socket.

2.4 The Client-Server Architecture

Socket programming typically follows a **client-server model**. It is crucial to remember that "client" and "server" refer to programs, not physical machines.

- **Server**: A program that listens on a known IP address and port, waiting for incoming connection requests.
- **Client**: A program that initiates a connection to a server by specifying the server's known IP address and port.

A single host can run many client and server programs simultaneously. Now that we understand the basic components, we must recognize that sockets can be configured in different ways to support different communication protocols.

—

3 A Comparison of Socket Types: UDP vs. TCP

The choice between socket types fundamentally dictates the behavior and reliability of your network communication. The two most common types used in internet applications are Datagram sockets, which use the **User Datagram Protocol (UDP)**, and Stream sockets, which use the **Transmission Control Protocol (TCP)**.

Table 1: Comparison of Socket Types

Socket Type (Macro)	Protocol	Key Characteristics & Use Cases
`SOCK_DGRAM`	UDP	**Characteristics:** Connectionless, fast, and **unreliable**. Packets (datagrams) can arrive out of order, but if a packet does arrive, its contents are correct. **Use Cases:** Audio/video streams, multiplayer games.
`SOCK_STREAM`	TCP	**Characteristics:** Provides two-way, **connected** communication streams. It is **reliable**, handling packet reordering, retransmissions, and ensuring data is error-free. **Use Cases:** File transfer (FTP), HTTP, SSH.

Another type, RAW sockets, provides very powerful, low-level network access but is beyond the scope of these introductory notes. To implement either UDP or TCP sockets, we must first include and understand the specific libraries and functions provided by the system.

—

4 The C/C++ Socket Programming Toolkit

This section serves as a reference for the essential C and C++ header files and foundational functions required to create, configure, and manage sockets before any data can be exchanged.

The following standard C library headers provide the core structures and functions for socket programming:

- `<sys/socket.h>`: Provides the fundamental `socketaddr` struct and macros for socket types, such as `SOCK_DGRAM` and `SOCK_STREAM`.
- `<netinet/in.h>`: Contains the `socketaddr_in` struct, which is used specifically for storing internet protocol (IPv4) addresses.

- `<arpa/inet.h>`: Includes the `in_addr` struct and provides crucial functions for converting addresses between text and binary formats (e.g., `htonl`, `inet_pton`).
- `<unistd.h>`: Provides POSIX operating system API wrappers for standard I/O operations, including `read`, `write`, and `close`.
- `<string.h>`: Includes memory manipulation functions, such as `memset`, which are essential for initializing data structures.
- `<netinet/tcp.h>`: Defines the `tcphdr` struct and TCP-specific macros.

4.1 Core Functions

4.1.1 `socket()`

This function creates a socket and returns a file descriptor. It takes three parameters: `family` (e.g., `AF_INET` for IPv4), `type` (e.g., `SOCK_DGRAM` or `SOCK_STREAM`), and `protocol` (typically set to 0 to use the default protocol for the chosen type).

```
// Create a UDP socket
int udp_socket_fd = socket(AF_INET, SOCK_DGRAM, 0);

// Create a TCP socket
int tcp_socket_fd = socket(AF_INET, SOCK_STREAM, 0);
```

4.1.2 `bind()`

This function associates a socket with a local address and port number. A server uses `bind()` to specify which port it will listen on. Using the constant `INADDR_ANY` allows the socket to accept connections on all available network interfaces (e.g., Wi-Fi, Ethernet).

```
int sckfd = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in my_addr = {0}; // Initialize struct to all zeros
my_addr.sin_family = AF_INET; // Family IPv4
my_addr.sin_port = htons(recv_port); // Set socket port
my_addr.sin_addr.s_addr = htonl(INADDR_ANY); // Bind to all interfaces

// Bind the socket and check for errors
if (bind(sckfd, (struct sockaddr*) &my_addr, sizeof(my_addr)) < 0) {
    // ERROR: bind failed, exit or handle error
}
```

In this example, `(struct sockaddr*) &my_addr` is a crucial type cast. The `bind()` function expects a pointer to a generic `sockaddr` struct, but we are providing a more specific `sockaddr_in` struct for IPv4. The cast allows our specific struct to be used by the generic function. The `if` statement checks the return value of `bind()`; a negative value indicates an error, which is common if the port is already in use.

4.1.3 `close()` and `shutdown()`

These functions are used to terminate a connection, but they behave differently.

- `close()`: Immediately destroys the socket file descriptor. Any pending data that has not been sent or received is lost.
- `shutdown()`: Offers a more graceful way to close a connection by providing granular control. It takes a flag to specify which communication channels to close:
  - `SHUT_RD`: Disables further receives.
  - `SHUT_WR`: Disables further sends.

  - `SHUT_RDWR`: Disables both sends and receives. This allows a program to signal that it will send no more data (`SHUT_WR`) while still being able to receive pending data already sent by the peer before the socket is fully closed.

4.2 Address and Byte Order Manipulation

The `sockaddr_in` struct is used to handle internet addresses. Proper manipulation requires converting between the computer's native byte order ("host byte order") and a standardized **network byte order**. Different computer architectures (e.g., Intel's little-endian vs. ARM's big-endian) store multi-byte integers differently. Network protocols mandate a single standard—network byte order, which is **big-endian**—to ensure all machines can interpret data correctly.

- **Byte-Order Conversion**: The functions `htonl()` (host-to-network-long), `htons()` (host-to-network-short), `ntohl()` (network-to-host-long), and `ntohs()` (network-to-host-short) are used to perform these critical conversions.
- **String/Address Conversion**: Functions are also needed to convert between human-readable IP address strings (e.g., "192.168.100.1") and their binary representation.
  - `inet_pton()`: The modern, preferred function that converts a string to a network address. It supports both IPv4 and IPv6.
  - `inet_ntoa()`: Converts a network address back into a string.
  - `inet_addr()`: An older function similar to `inet_pton` but only for IPv4.

```
// Modern approach: convert string to network address using inet_pton
if (inet_pton(AF_INET, "192.168.100.1", &dest_addr.sin_addr) <= 0) {
    // Error handling: conversion failed
}

// Convert network address back to a string
char* ip_str = inet_ntoa(dest_addr.sin_addr);
```

With these general tools understood, we can now apply them to implement a UDP client and server.

—

5 Implementing UDP Sockets (The Datagram Model)

This section demonstrates the **connectionless** nature of UDP through a step-by-step breakdown of a simple client-server application. In the UDP model, data is sent in discrete packets, or **datagrams**, without establishing a persistent connection.

5.1 Key UDP Communication Functions

- `recvfrom()`: This function is used to receive data from a UDP socket. It blocks until a datagram arrives. Crucially, it also populates a `sockaddr` struct (`srcaddr`) with the IP address and port of the sender, allowing the server to know who sent the message.
- `sendto()`: This function sends data to a specific destination. Because UDP is connectionless, the destination address and port (`dest_addr`) must be provided with every `sendto()` call.

5.2 UDP Server Implementation

The server's role is to create a socket, bind it to a well-known port, and then enter a loop to receive and process incoming datagrams.

5.2.1 Server Setup

The server creates a `SOCK_DGRAM` socket and binds it to a specific port on all available network interfaces (`INADDR_ANY`). The `setsockopt` call with `SO_REUSEADDR` is a good practice to allow the port to be reused immediately after the server is shut down.

```
int sckfd = socket(AF_INET, SOCK_DGRAM, 0);
int option = 1;
if (sckfd < 0){ /*ERR*/}
setsockopt(sckfd, SOL_SOCKET, SO_REUSEADDR, (char*)&option, sizeof(option));

struct sockaddr_in my_addr = {0};
struct sockaddr_in srcaddr;
socklen_t addrlen = sizeof(srcaddr);

my_addr.sin_family = AF_INET;
my_addr.sin_port = htons(recv_port);
my_addr.sin_addr.s_addr = htonl(INADDR_ANY);

// Bind the socket to the address and port
if (bind(sckfd, (struct sockaddr*) &my_addr, sizeof(my_addr)) < 0) {
    // Error handling
}
```

5.2.2 Data Exchange Loop

This `while` loop is the heart of the server. It continuously waits for a message using `recvfrom()`. When a message arrives, it prints the content and then uses `sendto()` to echo the same message back to the original sender, whose address was captured in the `srcaddr` struct by the `recvfrom` call.

```
char buffer[256] = {0};
size_t max_size = 256;
int size = 0;
while((size = recvfrom(sckfd, buffer, max_size, 0, (struct sockaddr *) &srcaddr, &addrlen)) > 0) {
  std::cout << buffer << std::endl; // Print what was received
  // Echo the data back to the sender
  if(sendto(sckfd, buffer, size, 0, (struct sockaddr*) &srcaddr, addrlen) < 0)
  { break; } // If error in send, exit the while
  memset(buffer, 0, max_size); // Clear the buffer for the next message
}
```

5.3 UDP Client Implementation

The client creates a socket but does not need to bind it. Its primary task is to construct a `sockaddr_in` struct containing the server's IP address and port and then use it to send data.

5.3.1 Client Setup

The client creates a `SOCK_DGRAM` socket and prepares a `dest_addr` struct with the server's information. The `inet_pton` function is used to convert the server's IP address from a string to the required network format.

```
int sckfd = socket(AF_INET, SOCK_DGRAM, 0);
if (sckfd < 0){ /*ERR*/}
int option(1);
setsockopt(sckfd, SOL_SOCKET, SO_REUSEADDR, (char*)&option, sizeof(option));

struct sockaddr_in dest_addr = {0};
dest_addr.sin_family = AF_INET;
const char* dest_ip = "192.168.100.12";
```

```
int dest_port = 55555;
dest_addr.sin_port = htons(dest_port);

if (inet_pton(AF_INET, dest_ip, &dest_addr.sin_addr) <= 0) {
  /* Conversion of host_ip to AF_ADDRESS FAILED */
}
```

5.3.2 Data Exchange Loop

The client sends an initial message to the server using `sendto()`. It then blocks on `recvfrom()` to receive the echo reply from the server before printing it and repeating the loop.

```
char buffer[256] = "ciao";
size_t max_size = 256;
while(sendto(sckfd, buffer, max_size, 0, (struct sockaddr*) &dest_addr, sizeof(dest_addr )) > 0) {
  memset(buffer, 0, max_size); // Clear buffer to prepare for receiving
  socklen_t addrlen = sizeof(dest_addr);
  // Wait to receive the echo from the server
  if(recvfrom(sckfd, buffer, max_size, 0, (struct sockaddr *) &dest_addr, &addrlen) < 0)
    { break; } // If error in receive, exit the while
  std::cout << buffer << std::endl; // Print the received echo
}
```

UDP's simplicity contrasts sharply with the more complex, stateful nature of TCP, which guarantees reliable, in-order delivery of data.

—

6 Implementing TCP Sockets (The Stream Model)

Unlike UDP, TCP is a **connection-oriented** protocol. This requires an explicit setup process where a server listens for connections and a client actively connects to it. Once this connection is established, data can be exchanged reliably as a continuous stream.

6.1 Key TCP Communication Functions

The sequence of operations for TCP is more structured than for UDP.

6.1.1 Server-Side Sequence

1. `listen()`: After a socket is created and bound, `listen()` marks it as a passive listening socket. This function also sets a limit on the queue for pending incoming connections.
2. `accept()`: This is a blocking call that waits for a client to initiate a connection. When a client connects, `accept()` creates and returns a **new** socket file descriptor (`sockfd`). This new socket is dedicated exclusively to communication with that specific client, allowing the original listening socket to continue accepting other connections.

6.1.2 Client-Side Function

1. `connect()`: The client uses this function to initiate a connection to the server's listening socket.

6.1.3 Data Transfer Functions

Both `recv`/`send` and `read`/`write` can be used for TCP data transfer. While `read` and `write` are general-purpose POSIX I/O functions, `recv` and `send` are socket-specific. They offer more control via a `flags` parameter, though for basic stream communication they are functionally equivalent.

- `recv()` / `read()`: Used to receive data from a connected TCP socket.

- `send()` / `write()`: Used to send data over a connected TCP socket.

6.2 TCP Server Implementation

The TCP server is responsible for setting up a listening socket and then accepting client connections.

6.2.1 Setup Listening Socket

The server creates a `SOCK_STREAM` socket, binds it to a port, and prepares it for incoming connections.

```
int scklist = socket(AF_INET, SOCK_STREAM, 0);
if (scklist < 0){ /*ERR*/}
int option(1);
setsockopt(scklist, SOL_SOCKET, SO_REUSEADDR, (char*)&option, sizeof(option));

struct sockaddr_in my_addr = {0};
my_addr.sin_family = AF_INET;
int listen_port = 55555;
my_addr.sin_port = htons(listen_port);
my_addr.sin_addr.s_addr = htonl(INADDR_ANY);

if (bind(scklist, (struct sockaddr*)&my_addr, sizeof(my_addr)) < 0) { /*err*/}
```

6.2.2 Listen and Accept Connection

This is the most critical part of TCP server logic. The server calls `listen()` to start waiting for clients. The `accept()` call blocks until a client connects. Upon connection, it creates a ***new socket*** (`sockfd`) just for this client's conversation. The original socket (`scklist`) remains open and continues listening for other clients.

```
// Set the socket to listen for incoming connections, with a queue limit of 5
if (listen(scklist, 5) < 0) {
  //ERR
}

struct sockaddr_in client_addr;
socklen_t addr_l = sizeof(client_addr);
// Block and wait for a client to connect, creating a new socket for communication
int sockfd = accept(scklist, (struct sockaddr*) &client_addr, &addr_l);
if(sockfd < 0) {
  /*ERROR! CLOSE AND EXIT*/
}

// At this point, `scklist` is for listening, `sockfd` is for communicating
std::cout << "Connection from " << inet_ntoa(client_addr.sin_addr) << std::endl;
```

6.2.3 Data Exchange and Closure

Data is exchanged using `recv()` and `send()` on the new `sockfd`. Once communication is complete, *both* the client-specific socket (`sockfd`) and the original listening socket (`scklist`) must be closed.

```
char buf[256] = {0};
size_t max_size = 256;
int rcv_size = recv(sockfd, buf, max_size, 0);
if(rcv_size < 0) {
   /*ERROR! CLOSE AND EXIT!!!*/
}

std::cout << buf << std::endl;
int sent_size = send(sockfd, buf, rcv_size, 0);
```

```
if(sent_size < 0) {
  /*ERROR: CLOSE AND EXIT!*/
}

close(sockfd);  // Close the communication socket
close(scklist); // Close the listening socket
```

6.3 TCP Client Implementation

The TCP client creates a socket and actively connects to the server before any data can be exchanged.

6.3.1 Client Setup

The client creates a `SOCK_STREAM` socket and populates a `dest_addr` struct with the server's IP and port, similar to the UDP client.

```
int sckfd = socket(AF_INET, SOCK_STREAM, 0);
if (sckfd < 0){ /*ERR*/}
int option(1);
setsockopt(sckfd, SOL_SOCKET, SO_REUSEADDR, (char*)&option, sizeof(option));

struct sockaddr_in dest_addr = {0};
dest_addr.sin_family = AF_INET;
const char* dest_ip = "192.168.100.12";
int dest_port = 55555;
dest_addr.sin_port = htons(dest_port);

if (inet_pton(AF_INET, dest_ip, &dest_addr.sin_addr) <= 0) {
  /* Conversion host_ip to AF_ADDRESS FAILED */
}
```

6.3.2 Connect and Exchange Data

The client calls `connect()` to establish the connection. If successful, it can immediately use `send()` and `recv()` to communicate with the server over the established stream.

```
if (connect(sckfd, (struct sockaddr*) &dest_addr, sizeof(dest_addr)) < 0) { /*ERROR: CLOSE AND EXIT*/}

char buf[512] = "data to tx";
size_t data_size = 10;
int sent_size = send(sckfd, buf, data_size, 0);
if(sent_size < 0) {/*ERROR: CLOSE AND EXIT*/}

memset(buf, 0, 512); // Clear buffer for receiving
int rcv_size = recv(sckfd, buf, 512, 0);
if(rcv_size < 0) {/*ERR!!!*/}

std::cout << buf << std::endl;
close(sckfd);
```

Building upon these implementations, we now turn to topics relevant to both TCP and UDP programming, such as robust error handling and proper buffer management.

—

7 Advanced Topics & Best Practices

Writing functional network code is only the first step. This section covers a collection of crucial, real-world considerations for creating robust, efficient, and stable network applications, including buffer

management, socket configuration, and critical signal handling.

7.1 Buffer Management

Buffers are required for both sending and receiving data through sockets. While traditional C-style arrays (`char buffer[SIZE]`) are simple, modern C++ offers safer solutions that prevent common memory errors. The best approach is to use **smart pointers** with `std::array`. This is superior because it provides automatic memory management (preventing leaks common with `new char[]`) while still offering direct, C-style array access via the `.data()` member function, which is required by the C-based socket APIs.

```
// Create a smart pointer to a std::array buffer
auto b1 = std::make_shared<std::array<char, 256>>();

// The ->data() member function provides the raw char* needed by socket calls
// e.g., recv(sckfd, b1->data(), b1->size(), 0);
```

7.2 Configuring Socket Options

The `setsockopt()` function allows you to modify the default behavior of a socket. A common and practical example is setting the `SO_REUSEADDR` option. After a TCP socket closes, the operating system keeps its port in a `TIME_WAIT` state for a few minutes to ensure any stray packets are handled correctly. This can prevent a server from restarting quickly on the same port. `SO_REUSEADDR` allows the kernel to reuse the port immediately, which is extremely useful during development.

```
int option(1); // Set option to true
int res = setsockopt(socketfd, SOL_SOCKET, SO_REUSEADDR, (char*)&option, sizeof(option));
```

7.3 Handling the SIGPIPE Signal

7.3.1 The Problem

The `SIGPIPE` signal is an asynchronous notification sent to a process when it attempts to write to a pipe or socket that is no longer readable (e.g., the remote peer has closed the connection). The default behavior for a process that receives `SIGPIPE` is to **terminate immediately**. For a server, this is catastrophic, as a single client disconnection could crash the entire service.

7.3.2 Solution 1: Ignoring the Signal

The simplest solution is to instruct the operating system to ignore `SIGPIPE`. This is typically done once during application initialization. When the signal is ignored, the `send()` or `write()` call that triggered the condition will fail gracefully, returning `-1`, and the global `errno` variable will be set to `EPIPE`. Your code can then check for this error and handle the disconnected client appropriately without crashing.

```
// This setup is typically run once when the application starts
#include <signal.h>
#include <cstring>

struct sigaction act;
memset(&act, '\0', sizeof(act));
act.sa_handler = SIG_IGN; // Set the handler to ignore the signal
if (sigaction(SIGPIPE, &act, NULL) < 0) {
    // Error setting signal handler
}
```

7.3.3 Solution 2: Using a Custom Handler

A more flexible approach is to register a custom signal handler function. This allows the program to perform specific cleanup actions, log the event, or manage client connections in a more sophisticated way when a `SIGPIPE` is caught.

```
#include <signal.h>
#include <cstring>

// A custom function to handle the signal
void handleIt(int sig_id) {
    // Logic to handle the broken pipe, e.g., logging
}

// Register the custom handler at application startup
struct sigaction act;
memset(&act, '\0', sizeof(act));
act.sa_handler = &handleIt; // Point to our custom function
if (sigaction(SIGPIPE, &act, NULL) < 0) {
    // Error setting signal handler
}
```

Finally, to complement code-based debugging, we will review some practical command-line tools for testing network connectivity.

—

8 Practical Networking Utilities

Command-line tools are invaluable for debugging and testing socket-based applications without needing to write a full client or server program each time.

8.1 `ping`

The `ping` utility is used to check the reachability of a host on a network and measure the round-trip time for messages. It is the most basic tool for confirming that a remote machine is online and responsive.

```
ping 147.162.97.5
```

8.2 `netcat` (`nc`)

`netcat` is a versatile utility for reading from and writing to network connections using either TCP or UDP. It can act as both a client and a server, making it an excellent tool for quick tests.

Key command flags include:

- `-l`: Listen mode (act as a server).
- `-u`: Use UDP protocol (default is TCP).

Examples:

```
# UDP Server listening on port 55556
nc -lu 55556

# UDP Client connecting to 147.162.97.6 on port 55556
nc -u 147.162.97.6 55556

# TCP Server listening on port 55555
nc -l 55555

# TCP Client connecting to 147.162.97.6 on port 55555
```

```
nc 147.162.97.6 55555
```

A Note on Firewalls: When testing network code, be aware that local or network **firewalls** may block traffic on arbitrary ports. For lab or educational environments, specific ports (like 55555 in our examples) may be explicitly opened for testing. If you experience connection issues, always check for firewall rules on both the client and server machines.